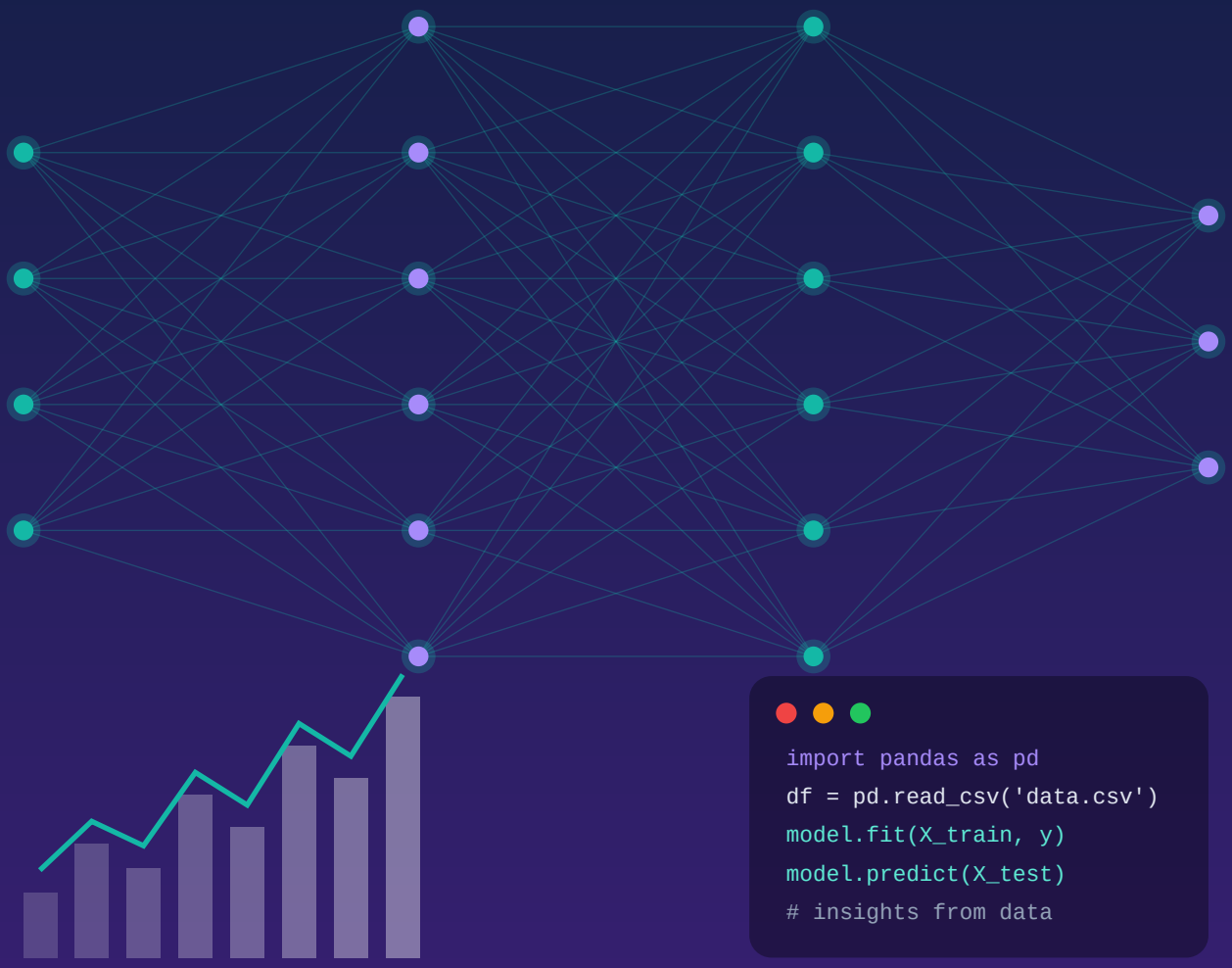

```
import pandas as pd
df = pd.read_csv('data.csv')
model.fit(X_train, y)
model.predict(X_test)
# insights from data
```

Data Science with AI

A beginner-friendly guide to the core skills, from Python to deployment

PYTHON | STATISTICS | PANDAS | SQL | MACHINE LEARNING | DEPLOYMENT

CONTENTS

What you will learn

Data science combines programming, mathematics, and domain knowledge to turn raw data into decisions. AI tools now speed up every stage, but the fundamentals below are what make you able to check and trust the results.

1 **Python Programming**
Syntax, functions, OOP, virtual environments

2 **Statistics and Math**
Probability, distributions, hypothesis tests, linear algebra

3 **Data Analysis with Pandas**
Cleaning, joins, groupby, EDA, visualisation

4 **SQL for Data Science**
Queries, joins, window functions

5 **Machine Learning**
Regression, classification, clustering, evaluation

6 **Projects and Deployment**
Streamlit apps, GitHub portfolio, AI-assisted workflow

CHAPTER 1

Python Programming

Syntax, functions, OOP, virtual environments

Python is the main language of data science thanks to its readable syntax and huge ecosystem of libraries such as NumPy, Pandas, and scikit-learn.

Syntax

- **Variables and types:** int, float, str, bool, plus collections like list, tuple, dict, and set.
- **Control flow:** if/elif/else, for and while loops, and list comprehensions.
- **Indentation** defines code blocks, so consistent spacing matters.

Functions

- Defined with **def**; they take parameters, return values, and can have defaults.
- **lambda** creates small anonymous functions, handy with map, filter, and sorting.
- Good functions do one job and have clear names.

Object-Oriented Programming (OOP)

- A **class** is a blueprint; an **object** is an instance of it.
- Key ideas: attributes, methods, inheritance, and encapsulation. scikit-learn models are classes with fit() and predict().

Virtual Environments

- Isolated spaces so each project has its own library versions.
- Create with **python -m venv env**, then install packages with **pip** and record them in requirements.txt.

Quick example

```
def mean(values):
    return sum(values) / len(values)

class Dataset:
    def __init__(self, name, values):
        self.name, self.values = name, values
    def summary(self):
        return f"{self.name}: mean={mean(self.values):.2f}"

print(Dataset("Scores", [70, 85, 90]).summary())
```

Key takeaway: Write small reusable functions, organise code with classes when it grows, and always work inside a virtual environment.

CHAPTER 2

Statistics and Math

Probability, distributions, hypothesis tests, linear algebra

Statistics and math explain why models work. They help you describe data, measure uncertainty, and avoid being fooled by random noise.

Probability

- Measures how likely an event is, from 0 (impossible) to 1 (certain).
- Core ideas: independence, conditional probability, and Bayes' theorem.

Distributions

- **Normal:** the bell curve, described by mean and standard deviation.
- **Binomial:** counts of successes in repeated yes/no trials. **Poisson:** counts of events in a time period.

Hypothesis Tests

- Compare a **null hypothesis** (no effect) with an alternative using data.
- The **p-value** shows how surprising your result would be if the null were true; below 0.05 is a common cut-off.
- Common tests: t-test, chi-square, ANOVA.

Linear Algebra

- **Vectors and matrices** are how data and model parameters are stored.
- Matrix multiplication, dot products, eigenvalues, and PCA power most ML algorithms.

Quick example

```
from scipy import stats
import numpy as np

group_a = np.array([72, 75, 78, 71, 74])
group_b = np.array([80, 82, 79, 85, 81])
t, p = stats.ttest_ind(group_a, group_b)
print(f"t={t:.2f}, p={p:.4f}")   # small p => groups differ
```

Key takeaway: A small p-value suggests a real difference, but always consider sample size and practical importance too.

CHAPTER 3

Data Analysis with Pandas

Cleaning, joins, groupby, EDA, visualisation

Pandas is the go-to library for tabular data. Its main structure, the DataFrame, works like a spreadsheet you can control with code.

Cleaning

- Handle missing values with **isna()**, **dropna()**, and **fillna()**.
- Remove duplicates, fix data types, and standardise text.

Joins

- **merge()** combines tables on shared keys (inner, left, right, outer).
- **concat()** stacks tables vertically or side by side.

Groupby

- Split data into groups, apply a calculation, and combine the results, such as average sales per region.

EDA and Visualisation

- Exploratory Data Analysis uses **describe()**, **info()**, and **value_counts()** to understand your data.
- Plot with Matplotlib, Seaborn, or Plotly: histograms, box plots, scatter plots, and heatmaps.

Quick example

```
import pandas as pd

df = pd.read_csv("sales.csv")
df = df.dropna().drop_duplicates()
orders = df.merge(customers, on="customer_id", how="left")
print(orders.groupby("region")["amount"].mean())
orders["amount"].hist()
```

Key takeaway: Spend most of your time cleaning and exploring. Good analysis depends on good data.

CHAPTER 4

SQL for Data Science

Queries, joins, window functions

SQL is the standard language for retrieving data from databases. Most real-world data lives in databases, so SQL is an essential skill.

Queries

- **SELECT** columns **FROM** a table, filtered with **WHERE**, sorted with **ORDER BY**.
- Summarise with **GROUP BY** and aggregates such as COUNT, SUM, AVG, and use **HAVING** to filter groups.

Joins

- **INNER JOIN** keeps matching rows only; **LEFT JOIN** keeps all rows from the left table.
- Joins link tables through keys such as customer_id.

Window Functions

- Calculate across related rows without collapsing them.
- Examples: **ROW_NUMBER()**, **RANK()**, running totals with **SUM() OVER**, and **LAG()** for previous-row comparisons.

Quick example

```
SELECT c.name,
       o.order_date,
       o.amount,
       SUM(o.amount) OVER (PARTITION BY c.id ORDER BY o.order_date) AS running_total
FROM orders o
JOIN customers c ON c.id = o.customer_id
WHERE o.amount > 0
ORDER BY c.name, o.order_date;
```

> **Key takeaway:** Do the heavy filtering and aggregating in SQL, then bring only the results into Python.

CHAPTER 5

Machine Learning

Regression, classification, clustering, evaluation

Machine learning lets computers find patterns in data and make predictions without being explicitly programmed for each rule.

Regression

- Predicts a **number**, such as house price or sales. Examples: linear regression, random forest regressor.

Classification

- Predicts a **category**, such as spam or not spam. Examples: logistic regression, decision trees, random forest.

Clustering

- **Unsupervised** learning that groups similar items without labels. Example: K-Means for customer segments.

Evaluation

- Split data into **train and test** sets to check performance on unseen data.
- Regression metrics: MAE, RMSE, R-squared. Classification metrics: accuracy, precision, recall, F1.
- Watch for **overfitting** (great on training data, poor on new data) and use cross-validation.

Quick example

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = RandomForestClassifier().fit(X_train, y_train)
print(accuracy_score(y_test, model.predict(X_test)))
```

> **Key takeaway:** Start with a simple baseline model, evaluate it honestly on unseen data, then improve step by step.

CHAPTER 6

Projects and Deployment

Streamlit apps, GitHub portfolio, AI-assisted workflow

Projects prove your skills. Deploying them shows you can deliver something people can actually use.

Streamlit Apps

- A Python library that turns scripts into interactive web apps with no front-end knowledge.
- Add sliders, file uploads, charts, and model predictions in a few lines; deploy free on Streamlit Community Cloud.

GitHub Portfolio

- Publish 3 to 5 solid projects with a clear **README**: problem, data, method, results, and how to run it.
- Use commits, branches, and a requirements.txt so others can reproduce your work.

AI-Assisted Workflow

- Use AI assistants to draft code, explain errors, suggest visualisations, and write documentation.
- Always **review and test** what AI produces, and verify numbers yourself.
- Never paste sensitive or private data into public tools.

Quick example

```
import streamlit as st
import pandas as pd

st.title("Sales Explorer")
file = st.file_uploader("Upload a CSV")
if file:
    df = pd.read_csv(file)
    st.dataframe(df.head())
    st.bar_chart(df.select_dtypes("number").mean())
```

> **Key takeaway:** AI makes you faster, but understanding the fundamentals lets you catch its mistakes. Ship projects, share them, and keep learning.